Práctica 3 – Programa para ordenar alfabéticamente una lista de palabras

Objetivo

Escribir un programa en lenguaje ensamblador para ordenar alfabéticamente una lista de 16 palabras de 15 caracteres como máximo. Almacenar la lista de palabras ordenadas en la memoria de datos, visualizarlas por el hiperterminal.

Formato de la Lista de Palabras

Las palabras a ordenar estarán inicialmente incluidas en 16 strings cada uno de ellos definidos con la directiva .asciz[1]. Para facilitar la comparación, todos los caracteres son letras minúsculas del alfabeto inglés. Se os proporciona el fichero main.S con una lista concreta. Como podéis observar en la lista, entre otras, aparece una palabra repetida, otra con un único carácter y otra con 15 caracteres (el máximo permitido).

Algoritmo

Para realizar la tarea de ordenar las palabras, el programa comparará dos a dos todas ellas con la ayuda de dos punteros (r4 y r5) que contendrán la dirección del primer carácter de cada una de las dos palabras a comparar estableciendo un "ganador" y comparando dicho ganador con la siguiente palabra hasta finalizar la lista. Concretamente, las etapas del programa serán las siguientes:

1. Copiar la lista desde la memoria de programa a la memoria de datos a partir de su primera dirección (0x40000000). Para ello el programa principal llamará a la subrutina **cpypalabra**.
2. Colocar el puntero r4 en el primer carácter de la primera palabra de la lista y el puntero r5 en el primer carácter de la segunda palabra.
3. Con los punteros en estas posiciones, llamar a la subrutina **cmppalabra** que compara las dos palabras apuntadas y devuelve en los bits de estado el resultado de dicha comparación.
4. Una vez se haya vuelto al programa principal, si el resultado de la comparación ha sido que la palabra apuntada por r5 es la "ganadora"[2], se intercambiarán sus posiciones en la lista de la memoria de datos llamando a la subrutina **exchpalabra**. En caso contrario, no se hace nada.
5. A continuación, se coloca el puntero r5 en la siguiente palabra y se repiten los puntos 3 y 4.
6. Así se procederá hasta que el puntero r5 alcance el final de la lista de palabras (dirección 0x40000100). En ese momento, en la primera posición de la lista se encuentra la palabra "ganadora" de toda la lista, es decir, la primera ordenada alfabéticamente.
7. Ahora, se coloca el puntero r4 en la segunda palabra de la lista, el puntero r5 en la tercera palabra y se vuelve a proceder de la misma manera. Cuando el puntero r5 haya recorrido toda la lista, ya se encontrará en la segunda posición de la lista la segunda palabra ordenada alfabéticamente.
8. Y así se van ordenando las palabras hasta que el puntero r4 apunta a la última palabra de la lista, que será la última ordenada alfabéticamente. El algoritmo ha finalizado y en los 256 bytes de la memoria de datos se encuentra la lista ordenada.
9. Para finalizar el algoritmo, se enviará al hiperterminal la lista ordenada, de modo que se visualice:

```
Programa que ordena alfabeticamente palabras de como maximo 15 caracteres.
Vamos a ordenar:
enero febrero marzo abril mayo junio xxxxxxxxxxxxxxx julio agosto septiembre y octubre noviembre diciembre mayo ahora
Lista ordenada:
abril agosto ahora diciembre enero febrero julio junio marzo mayo mayo noviembre octubre septiembre xxxxxxxxxxxxxxx y
```

[1] La directiva .asciz añade un carácter final al string. Este carácter es el NULL (0x00). De modo que cada palabra tendrá como máximo 16 caracteres.

[2] Es la situada en una posición anterior en el diccionario

Subrutinas

Comenzar el programa escribiendo 3 subrutinas, que realicen las siguientes tareas. Utilizar el convenio de paso de parámetros a subrutinas.

Subrutina **cpypalabra**

> Copia la lista de palabras en la memoria de datos, exactamente del mismo modo que se encuentra en la memoria de programa, es decir, reservando un espacio de 16 bytes para cada palabra y finalizando cada palabra con el carácter nulo (0x00).

Subrutina **cmppalabra**

> Compara las palabras apuntadas por r0 y r1 (las palabras finalizan con el carácter nulo 0x00). Devuelve en el registro de estado (flags C y Z) el resultado de la comparación.

Subrutina **exchpalabra**

> Intercambia las posiciones en la memoria de datos de las palabras apuntadas por los registros r0 y r1.

Antes de escribir el programa principal, se aconseja comprobar el funcionamiento de las subrutinas. Para ello, escribir en el programa principal un código muy sencillo que las llame y compruebe que funcionan.

Una vez comprobado el correcto funcionamiento de las subrutinas, escribir el código del programa principal **main** que defina el movimiento de los punteros. El programa main debe finalizar con la visualización de la lista ordenada por el hiperterminal y, como siempre, retornando al ISR de reset, concretamente a la etiqueta **_Halt:** (bucle infinito).